Intelligent Agent

Nirmala Shinde Baloorkar

Assistant Professor

Department of Computer Engineering

SOMAIYA VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

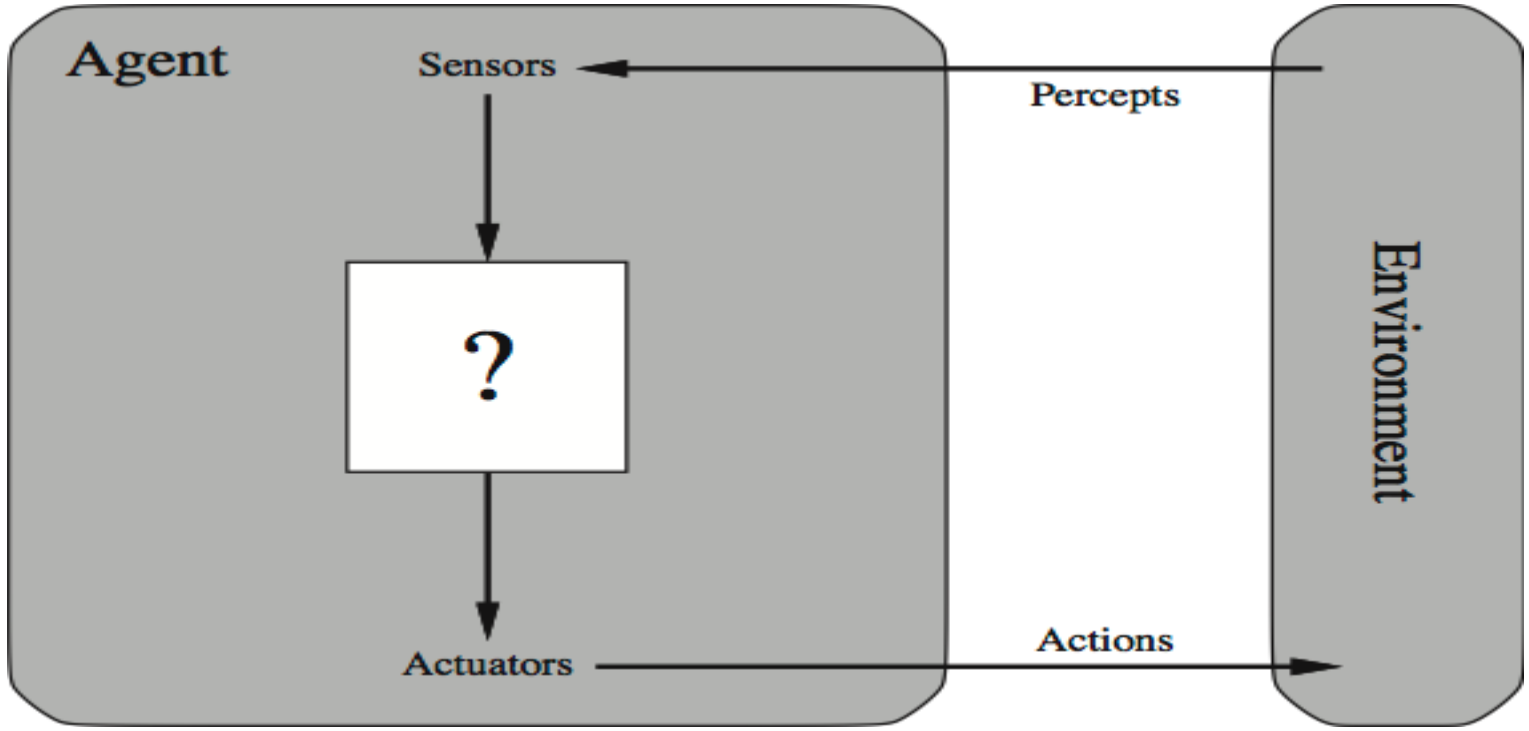
Somaiya TRUST

Outline

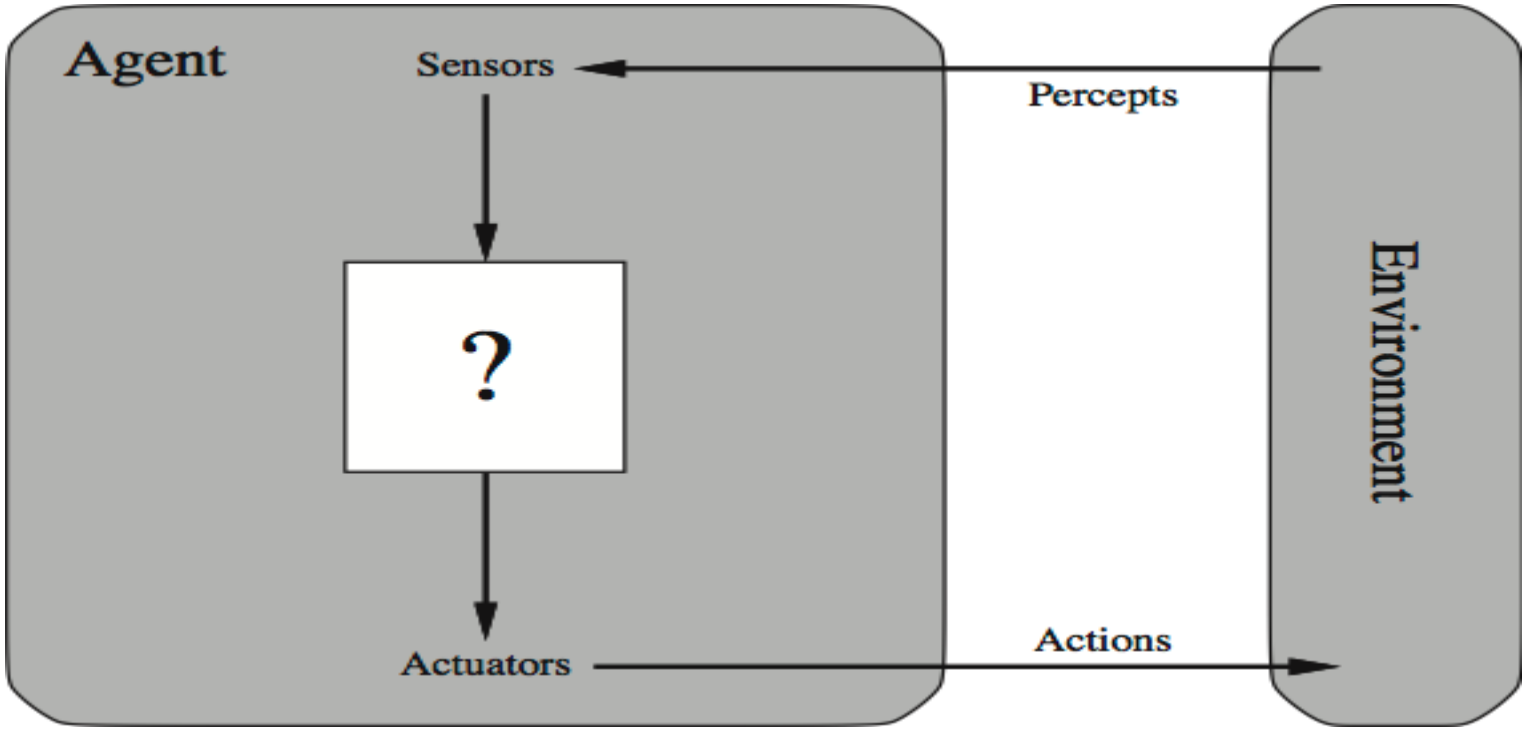
- Agent Type

Agent

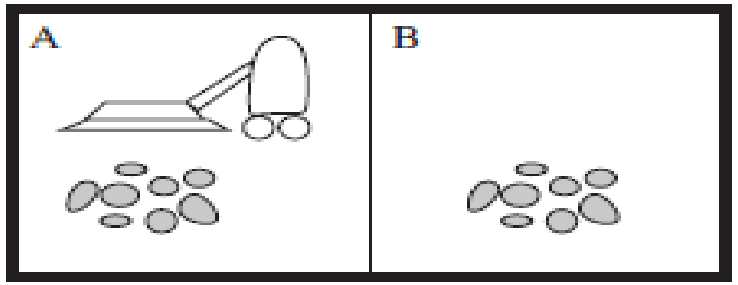
- It perceive the environment and act to achieve specific objectives.

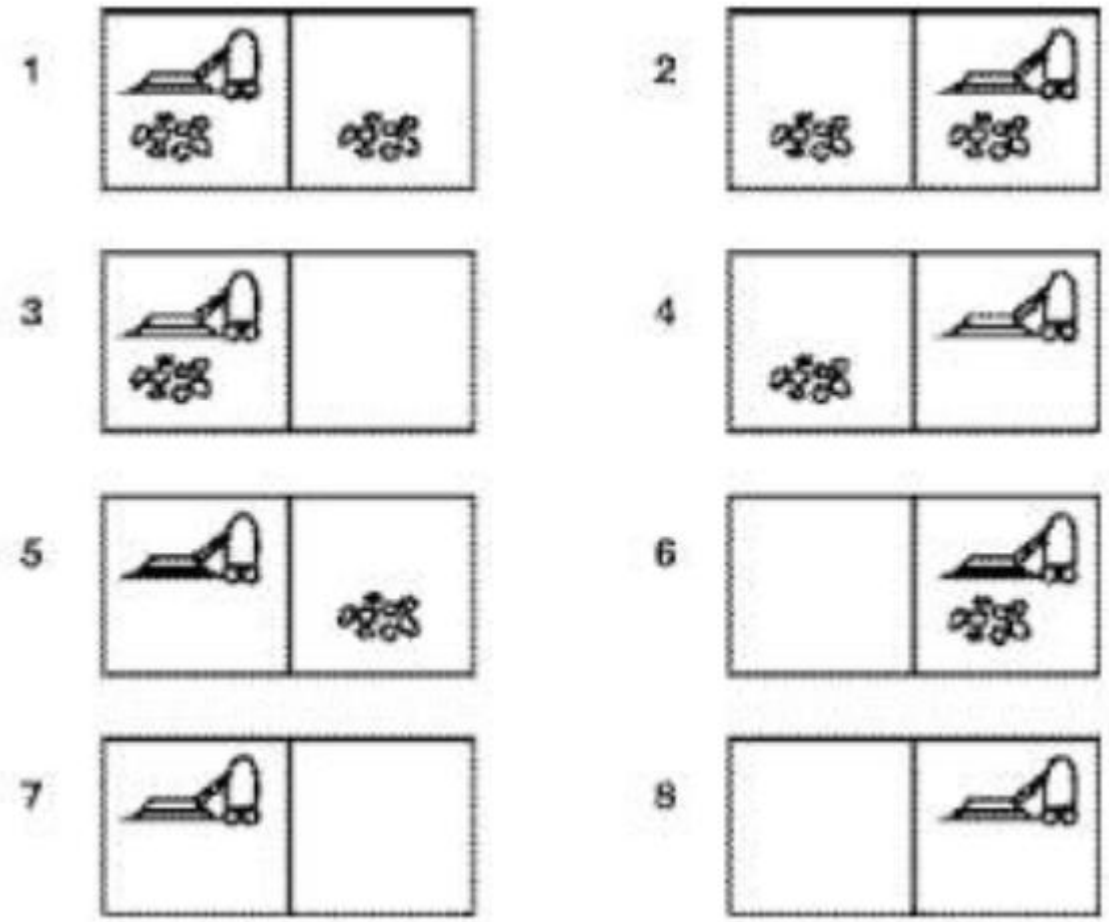


The structure of agent
agent = architecture + program



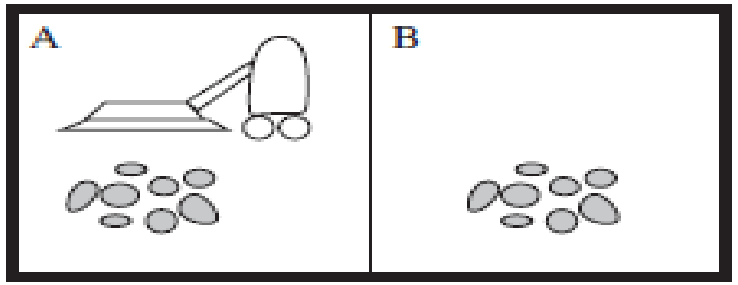
Vacuum cleaner World





- Percept: location and content e.g. {A, Dirty}

Vacuum cleaner World (contd)



- Percept: location and content e.g. {A, Dirty}
- Actions: Left, Right, Sweep, NoOp
- Agent Function: Mapping from precepts to actions.

Percept	Action
[A, Clean]	Right
[A, Dirty]	Sweep
[B, Clean]	Left
[B, Dirty]	Sweep

Agent types

- Basic types in order of increasing generality:
  - Table Driven Agent
  - Simple Reflex Agent
  - Model-based Reflex Agent
  - Goal-based Agent
  - Utility-based Agent
  - Learning Agent

Table Driven agent

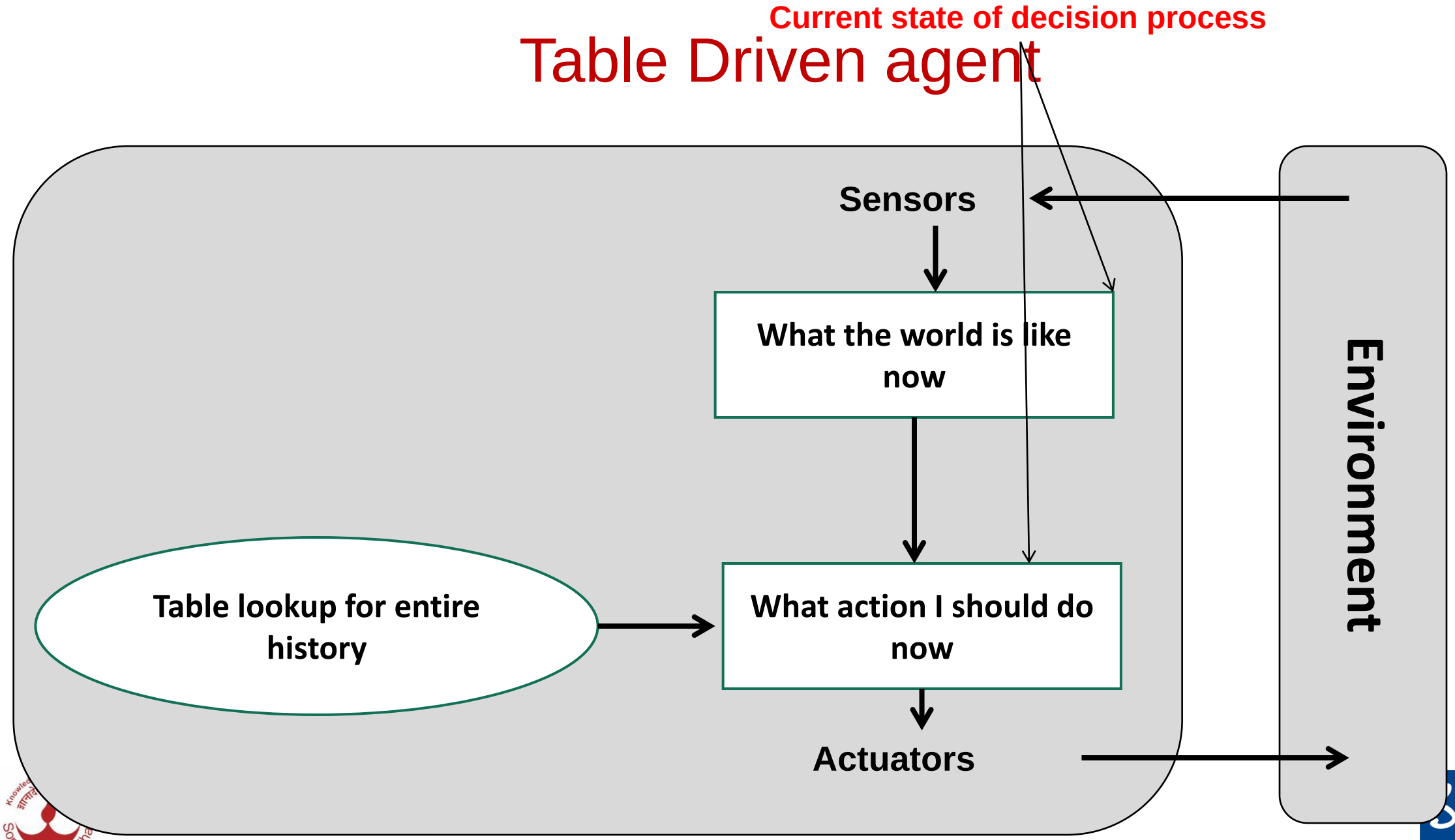


Table Driven agent (continued)

Function **TABLE-DRIVEN**(percepts)

returns an action

Input: Static : percepts, initially empty

table, initially fully specified

append *percept* to the end of percept sequence

action ← LOOKUP(percepts, table)

return action

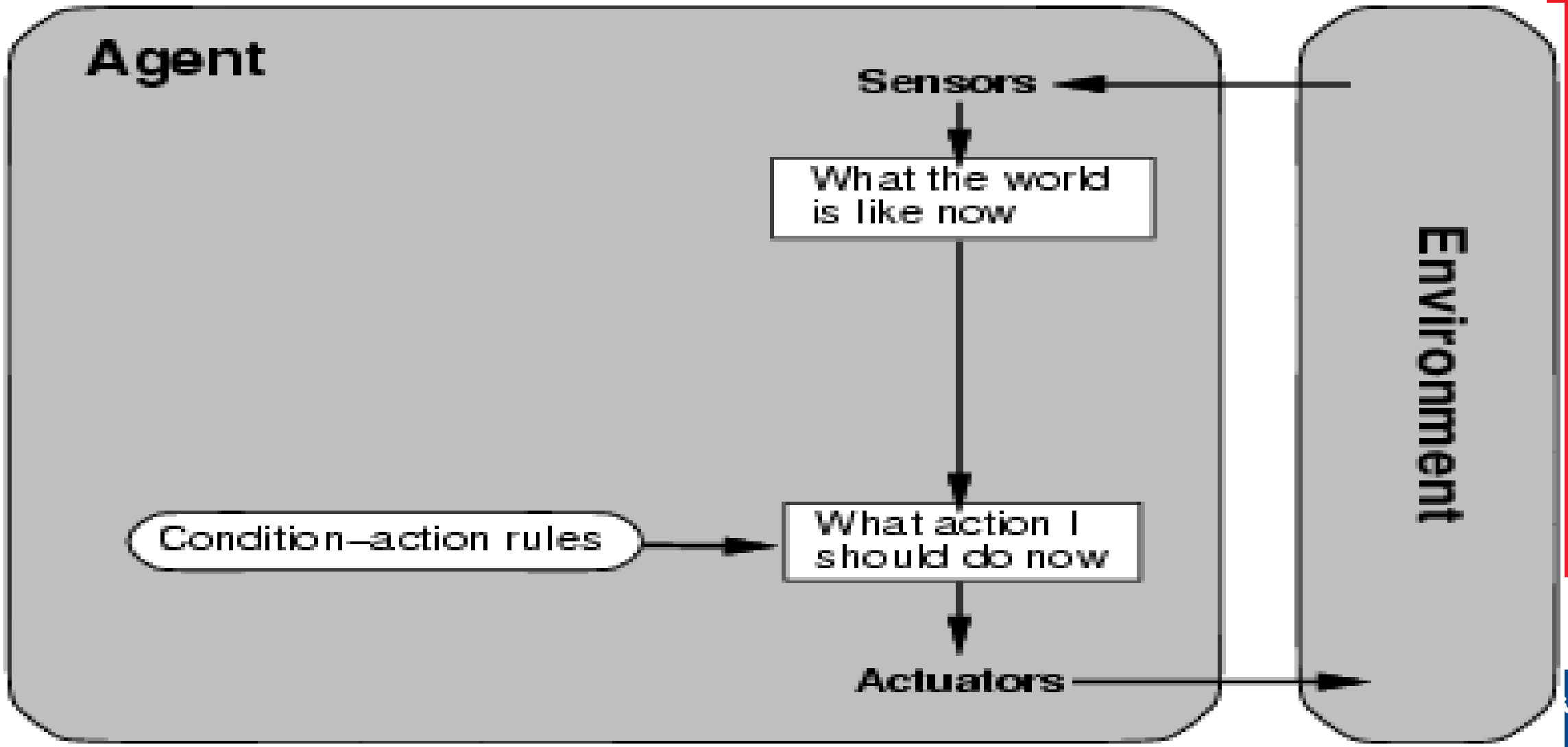
- It operates by keeping in memory its entire percept sequence.
- Uses it to index into table of actions (contains appropriate action for all possible sequences) to decide what to do.

Table-lookup agent

Drawbacks:

- Huge table
- Take a long time to build the table
- No agent could ever learn all the right table entries from its experience
- Even if the environment is simple enough to yield a feasible table size, the designer still has no guidance about how to fill in the table entries

Simple reflex agents



Simple reflex agents

- It is suitable for fully observable environment.
- It is capable of mapping the current percepts to the action & not based on past perception / action depends only on immediate percepts.
- Implement by condition-action rules

Simple reflex agents (continued)

- Condition-Action Rule (if….then)
  - It is a rule that maps a state(condition) to an action.
- E.g. automated taxi
  - If the car in front brakes and its brake lights come on.

if *car-in-front-is-braking* **then** *initiate-braking*.

Example –
Mail sorting robot

- Environment: Conveyor belt of letter (code),bin
- Rule
  - identify the destination of a parcel/letter etc.
    - City(Mumbai) → put KJSCE letter.
- Amazon using the robot which search items to ensure the right item gets shipped to the right customer.

https://www.youtube.com/watch?v=UtBa9yVZBJM

Simple Reflex Agent

function SIMPLE-REFLEX-AGENT(*percept*) **returns** action
  static: *rules*, a set of condition-action rules

  state ← INTERPRET-INPUT (*percept*)
  rule ← RULE-MATCH (*state*,*rules*)
  action ← RULE-ACTION [*rule*]
  return *action*

First match.
No further matches sought.
Only one level of deduction.

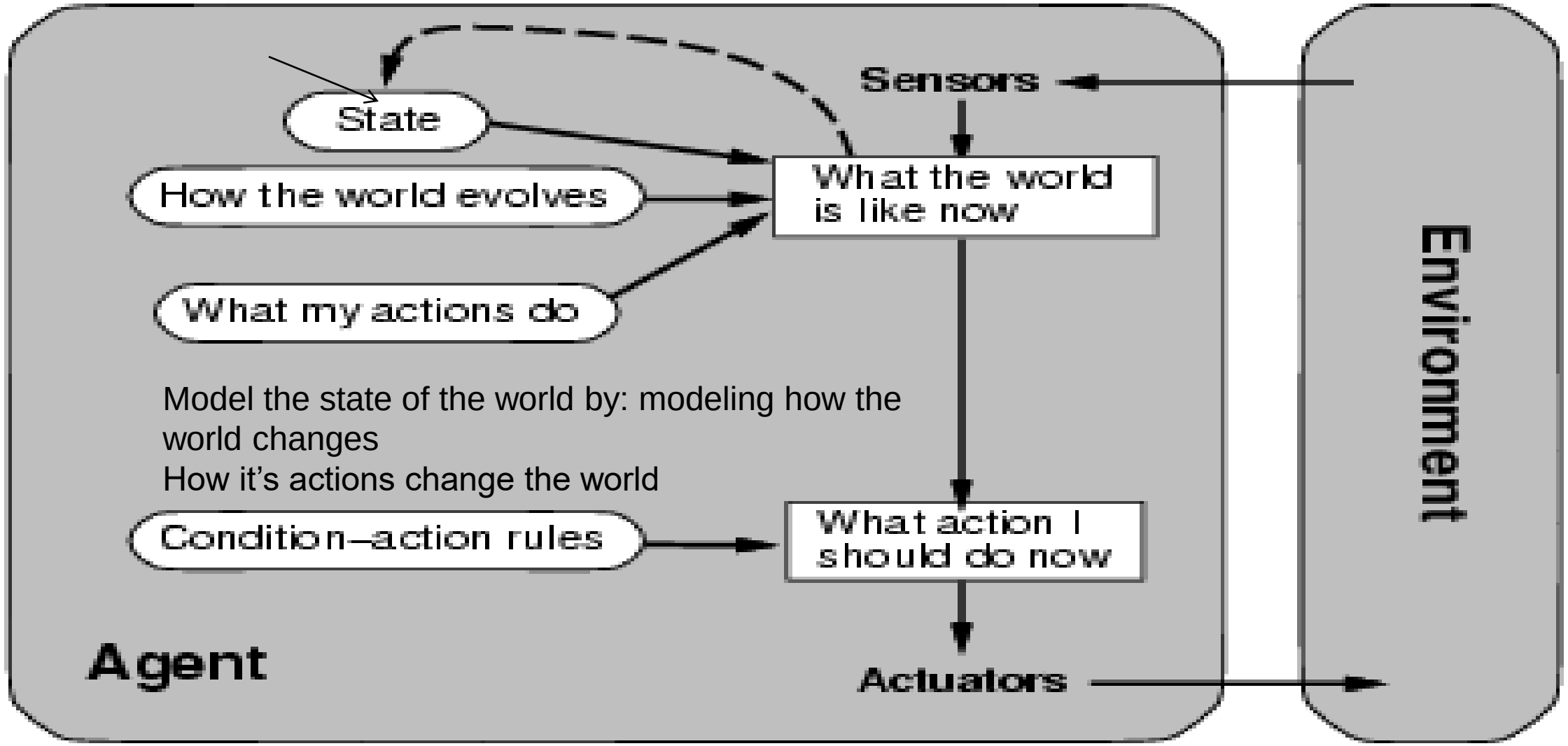
A simple reflex agent works by finding a rule whose condition matches the current situation (as defined by the percept) and then doing the action associated with that rule.

Simple reflex agent: limitations

- Work *only if the correct decision can be made on the basis of only the current percept, i.e. only if the environment is fully observable.*
- Even a little bit of unobservability can cause serious trouble
  - E.g single image may not be sufficient to know whether the car in front is braking.

Model-based reflex agents

Description of current world state



Model the state of the world by: modeling how the world changes
How it's actions change the world

Model-based reflex agents

- Handle Partial Observability by keeping track of the unseen part of world.
- Action may depend on history
- Need to maintain internal world model
- Internal state requires two kinds of knowledge
  - Information about how the world evolves independently of the agent.
  - How the agent's own actions affect the world

Why need internal state?

- For the braking car problem,
  - the internal state is not too extensive just the previous frame from the camera, allowing the agent to detect when two red lights at the edge of the vehicle go on or off simultaneously.
- For other driving tasks such as changing lanes,
  - the agent needs to keep track of where the other cars are if it can't see them all at once.
- Waymo's self-driving cars in Arizona
- Amazon Bedrock is a service that provides access to foundation models (FMs) from AI companies
- Click Here for Waymo

Agent Program for Model Reflex Agent

Function **Model-reflex(**percept**)** returns an action

Input: state,rules, action

State ← UPDATE-STATE(state,action,percept)

Rule← RULE-MATCH(state,rules);

Action← RULE-ACTION(rule);

Return action

*UPDATE-STATE updates the internal state of the agent function.

Model-based reflex agents (continued)

Drawbacks

- Creating and keeping models up-to-date can use a lot of computer power.
- These models might not fully represent how complicated the real world is.
- They also can’t predict every possible situation that could happen.

Goal based agent

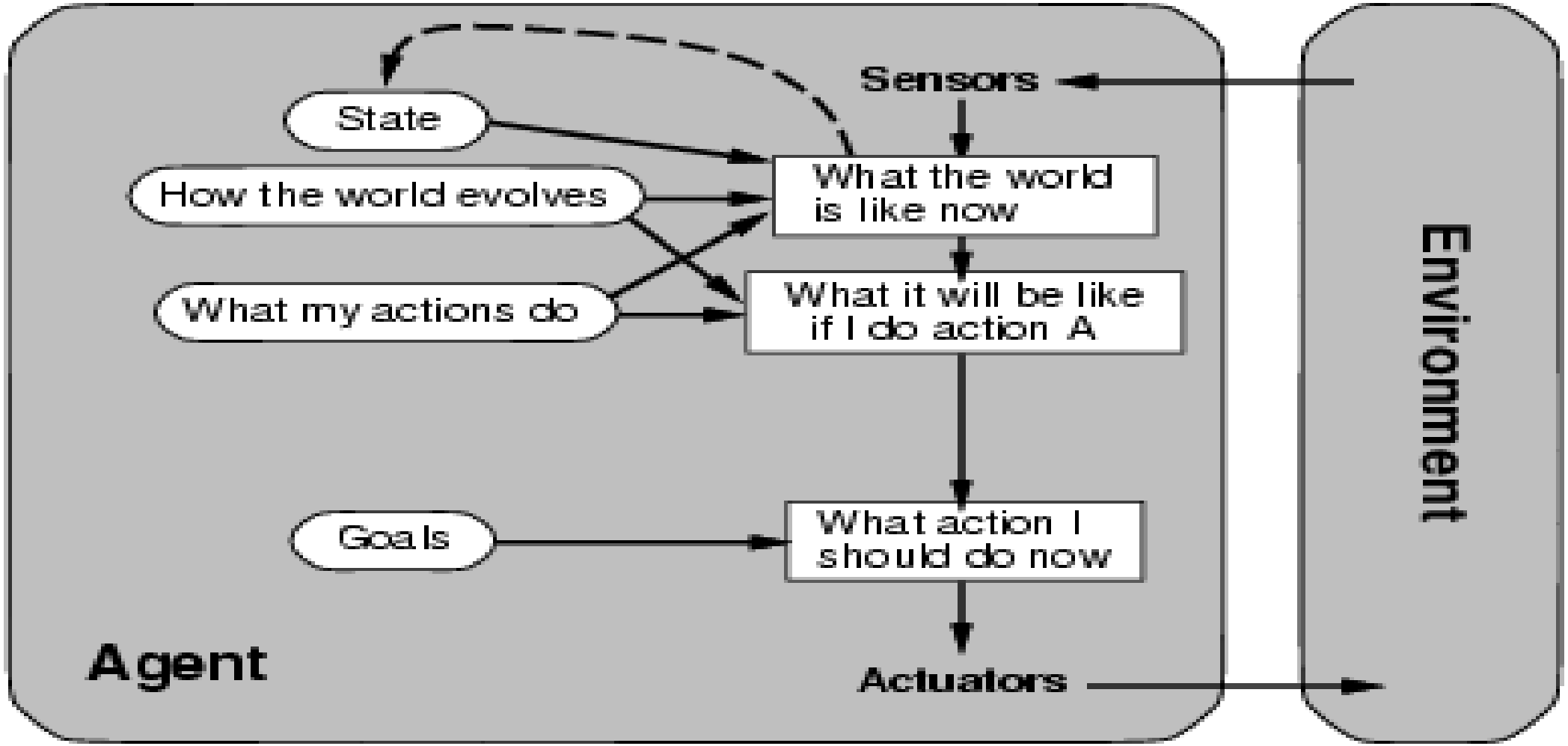
- Knowing something about the current state of the environment is not always enough to decide what to do.
- In this situation goal will help you to take the decision.

- If you are having specify goals, it makes the problem simpler.

- G+ Robot By China's Alibaba

Goal-based agents

Goals provide reason to prefer one action over the other.
We need to predict the future; we need to plan & search



Goal based agents

- Agent needs some sort of **goal** information that describes situations that are desirable
  - E.g.at a road junction, the taxi can turn left, turn right or go straight.
  - The correct decision depends on where the taxi is trying to get to.
- Combines action results with desired goals
- A goal condition may be satisfied just with one action or may take series of actions

Cont…

- More flexible because the knowledge that supports its decisions is represented explicitly and can be modified
  - E.g. in rainy season, the agent can update its knowledge of how effectively its brakes will operate; this will automatically cause all of the relevant behaviors to be altered to suit the new conditions

Goal based vs reflex based

- In reflex based, built-in rules map directly from percepts to actions
  - E.g. reflex agent brakes when it sees brake lights
- goal-based agent could reason given the way the world usually evolves and takes the only action that will achieve the goal
  - it will slow down to achieve the goal of not hitting other cars instead of apply brakes
- Reflex based have to rewrite many condition-action rules to solve new problem.
- The goal-based agent's behavior can easily be changed to go to a different location.

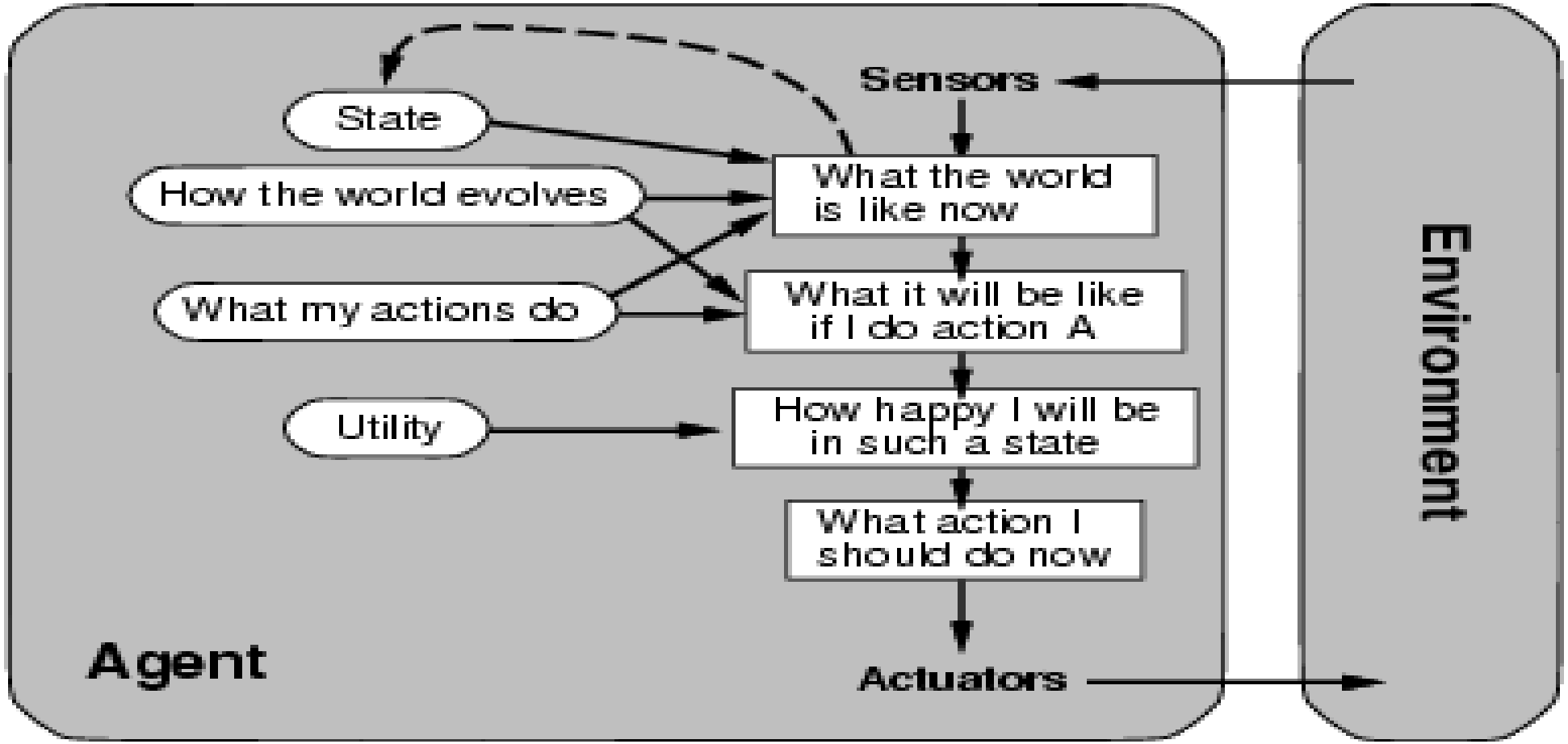
Categories of goal based agents

- Achieve goal - problem solving through searching
  - Uninformed search –BFS,DFS, DLS, IDS, Uniform Cost Search, Bidirectional search
  - Informed search-best first, A*, IDA*
  - Local search- Hill climbing, simulated annealing

- Achieve goal through planning
  - Partial order planning
  - Total order planning

Utility-based agents

Some solutions to goal states are better than others.
Which one is best is given by a utility function.
Which combination of goals is preferred?



Utility based agent

- Goals alone are not enough to generate high-quality behaviour in most environments.
- For example, taxi driver
  - Many action sequences will get the taxi to its destination (thereby achieving the goal)
  - But some are quicker, safer, more reliable or cheaper than others.

Utility based agent (continued)

- Goals just provide a binary distinction between “happy” and “unhappy” states.
- If agent achieve its goal then it is in happy state otherwise it is unhappy.
- “Happy” doesn’t sound scientific and economists, so we are using the term “Utility”

Utility functions

A **utility function** maps a state (or a sequence of states) onto a real number, which describes the associated degree of happiness. OR

The utility of an action is more if it satisfies the maximum constraints to achieve the goal.

- E.g. Travelling from Place A to Place B
  - Constraint are Time and Cost
  - Routes available are

1. Walking (>50 mins)
2. Train (<= 30mins)
3. Bus (>45mins)

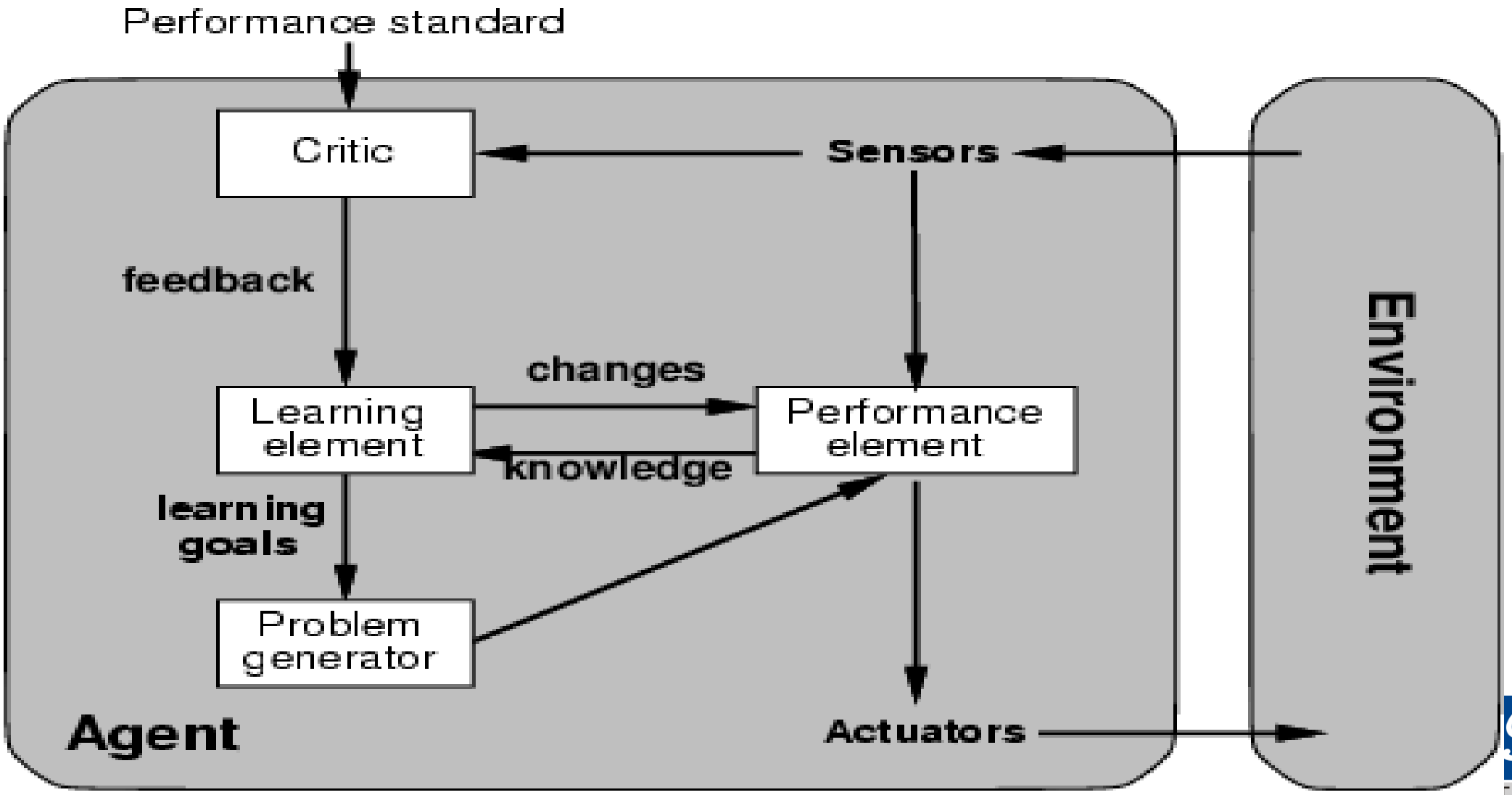
Utility based agent (continued)

- A complete utility function allows rational decisions in two kinds of cases where goals are inadequate.
  - First, when there are conflicting goals, only some of which can be achieved, the utility function specifies the appropriate tradeoff.
  - Second, when there are several goals that the agent can aim for, none of which can be achieved with certainty, utility provides a way in which the likelihood of success can be weighed up against the importance of the goals.

Learning agents

- Agents need to change their behavior to adapt to their environment.
- This requires an (external) teacher or reward which tell the agent how well it is currently doing.

Learning agents



Performance element

- Responsible for perceiving the environment deciding the action & executing the action.

OR

- Responsible for choosing actions that are known to offer good outcomes.
- E.g. Taxi Driver
  - collection of knowledge and procedures for selecting its driving actions.

critic

- Responsible for providing feedback
- Compares outcomes with some objective performance standard from outside the agent
- E.g. Taxi Driver
  - observes the world and passes information along to the learning element.

Learning element

- Responsible for improving the performance element
- Requires feedback on how well the agent is doing
- E.g. Taxi Driver
  - with i/p from critic, formulate a rule saying if actions was good or bad, and then modify performance element by installing the new rule.

problem generator

- Allows the agent to explore.
- Problem generator is responsible for suggesting actions that will lead to new and informative experiences.
- E.g Taxi Driver
  - trying out the brakes on different road surfaces under different conditions

Design of learning element

Three major issues:

- Which *components* of the performance element are to be learned.
- What *feedback* is available to learn these components.

(*What not to take as a feedback)

- What *representation* is used for the components.

Summary

- Simple reflex agent
  - Mobile keyboard suggestions
- Model based reflex agent
  - http://en.akinator.com/personnages/jeu
- Goal based agent
  - MYCIN
- Utility based agent
  - Taxi Driving agent
- Learning Agent

Problem statement

- Design an agent architecture for a pro-kabaddi referee. The referee is supposed to be on the field during match, announce the points as and when any team wins the same, give punishment cards to the players on misconduct and is an interface between the TV umpire and teams whenever the teams ask for reviews. The number of reviews are limited and the count will be kept by referee.

Note :The scores will be kept by another team and will be available on boards easily readable for viewers and referee.